



NEW PERSPECTIVES

HTML5, CSS3, and JavaScript 6th Edition

Tutorial 2

Getting Started with CSS

Objectives

- Explore the history of CSS
- Study different types of style sheets
- Explore style precedence and inheritance
- Apply color in CSS
- Use contextual selectors
- Work with attribute selectors

Objectives (continued)

- Apply text and font styles
- Use a web font
- Define list styles
- Work with margins and padding space
- Use pseudo-classes and pseudo-elements
- Insert page content with CSS

CSS Styles and Colors

The browser window background color is set to the color value `hsl(27, 73%, 72%)` using the `html` style rule.

The `h1` headings appear in white on a dark orange background as specified by the `h1` style rule.

Links

- [Home](#)
- [Running](#)
- [Cycling](#)
- [Swimming](#)
- [Active.com](#)
- [Runner's World](#)
- [endomondo.com](#)
- [Strava](#)
- [Bicycling Magazine](#)
- [VeloNews](#)
- [Bicycle Tutor](#)
- [Swim Smooth](#)
- [Swimming World](#)
- [USA Swimming](#)
- [triathlon.org](#)
- [usatriathlon.org](#)
- [Texas Triathlons](#)
- [CapTex Triathlon](#)
- [Triathlon Calendar](#)
- [Triathlete.com](#)
- [TriFuel.com](#)

About TSS

Since 2002, Tri and Succeed Sports has provided Austin with a first class training center for athletes of all abilities and goals. We specialize in helping you reach your full potential. You tell us what you want to do; we work to fulfill your needs.

Want to swim? Great! Interested in improving your cycling? Fantastic! Want to tackle a triathlon? We're there for you: before, during, and after the race. Or do you just want to get more fit? We are on it. We customize our instruction to match your goals. And you will finish what you start.

Classes

Winter instruction starts soon. Get a jump on your summer goals by joining us for individual or group instruction in:

- **Running:** We start with the basics to help you run faster and farther than you ever thought possible without aches and pains.
- **Cycling:** The indoor bike trainers at TSS include everything you need to refine your technique, stamina, and power for improved results on the road.
- **Swimming:** The open water swim can be one of the most frightening sports to master. Our classes begin with basic techniques so that your swim can be very enjoyable, and not a chore.

Contact us to set up individual instruction and assessment.

Our Philosophy

Athletes are the foundation of every successful training program. The best coach is an experienced guide who begins with each athlete's hopes, dreams and desires and then tailors a training plan based on that individual's current fitness and lifestyle. Since 2002, TSS has helped hundreds of individuals achieve success in many fitness areas. The winner is not the one who finishes first but anyone who starts the race and perseveres. Join us and begin exploring the possible.

Comments

Thank you for all that you have done. I am amazed at my progress. I realize that I have lofty goals but you have me well on my way.

Alison kept me focused working toward my dreams. She fosters a supportive and caring environment for growth as an athlete and as a person. Thank you!

You do it right! Your track record proves it. Proud to be a TSS athlete and I'm honored to have you all as my coaches and support team.

The coaches at TSS treat you with the highest respect: whether you're an individual getting off the couch for the first time or an elite athlete training for the Iron Man. They know their stuff.

The `h2` headings appear in white on a light orange background as specified by the `h2` style rule.

Page body background color is set to ivory using the `body` style rule.

Page text is set to the color value `rgb(91, 91, 91)`.

Introducing CSS

- The appearance of the page is determined by one or more style sheets written in the Cascading Style Sheets (CSS) language
- The latest version of the CSS language is **CSS3**
- CSS3 is built upon several **modules**, where each module is focused on a separate design topic

Types of Style Sheets

- **Browser styles or user agent styles** – Styles built into the browser
- **User-defined styles** – Styles defined by a user based on the configuration setting of the user's browser
- **External styles** – Styles created by a website author, placed within a CSS file, and linked to the page

Types of Style Sheets (continued)

- **Embedded styles** – Styles added to the head of an HTML document
- **Inline styles** – Styles added as element attributes within an HTML document and applied to only that particular element

Exploring Style Rules

- The general syntax of a CSS style rule is

```
selector{  
    property1: value1;  
    property2: value2;  
    ...  
}
```


Exploring Style Rules (continued)

- **Browser extensions** are an extended library of style properties in the browser
- **Vendor prefix** – Indicates the browser vendor that created and supports the style property

Vendor prefixes for browser extensions

Vendor Prefix	Rendering Engine	Browsers
-khtml-	KHTML	Konqueror
-moz-	Mozilla	Firefox, Camino
-ms-	Trident	Internet Explorer
-o-	Presto	Opera, Nintendo Wii browser
-webkit-	WebKit	Android browser, Chrome, Safari

Embedded Style Sheets

- They are inserted directly into the HTML file as metadata by adding the following element to the document head

```
<style>
```

```
    style rules
```

```
</style>
```

where *style rules* are the different rules embedded in the HTML page

Inline Styles

- They are styles applied directly to specific elements using the following **style** attribute

```
<element style="property1: value1;  
                property2: value2; ...">
```

content

```
</element>
```

where the *property: value* pairs define the styles applied directly to that element

Style Specificity and Precedence

- The more specific style rule has precedence over the more general style rule
- Specificity is an issue when two or more styles conflict
- If two rules have equal specificity and equal importance, then the one that is defined last has precedence

Style Inheritance

- **Style inheritance** – Process in which properties are passed from a parent element to its children
- For example, the following style rule sets the color of article text to blue and the rule is passed to any paragraph or other elements nested within that article

```
article {color: blue;}  
p {text-align: center;}
```

Browser Developer Tools

- They allow designers to view HTML code and CSS styles
- They make it easier to locate the source of a style that has been applied to a specific page element
- They are different in each browser and are updated and improved constantly

Writing Style Comments

Adding the @charset rule and style comments

the charset rule defines the character encoding used in the style sheet

```
@charset "utf-8";
```

CSS comments provide information about the style sheet

```
/*  
    New Perspectives on HTML5 and CSS3, 7th Edition  
    Tutorial 2  
    Tutorial Case  
  
    TSS Typographic Style Sheet  
    Author: Alison Palmer  
    Date: 2017-03-01  
  
    Filename: tss_styles.css  
*/
```

author name and current date

Importing Style Sheets

- **@import** is a CSS at-rule used to import the content of a style sheet file

```
@import url(url);
```

where *url* is the URL of an external stylesheet file

- It is similar to adding `link` elements to an HTML file

Working with Color in CSS

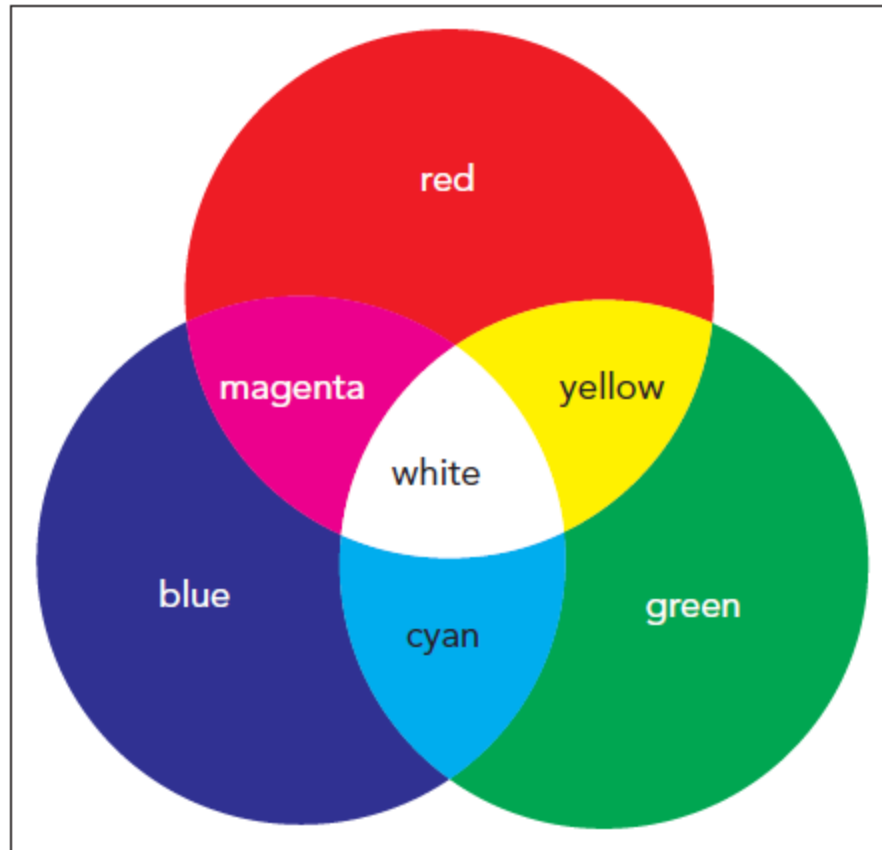
- **Color values** – Values in which the color is given by an exact numeric representation
- **RGB triplet** – The intensity of primary colors expressed as a set of numbers in CSS

`rgb (red, green, blue)`

- **Hexadecimal numbers** – A number expressed in the base 16 numbering system

RGB Color Values

Figure 2-8 Color addition in the RGB color model



© 2016 Cengage Learning

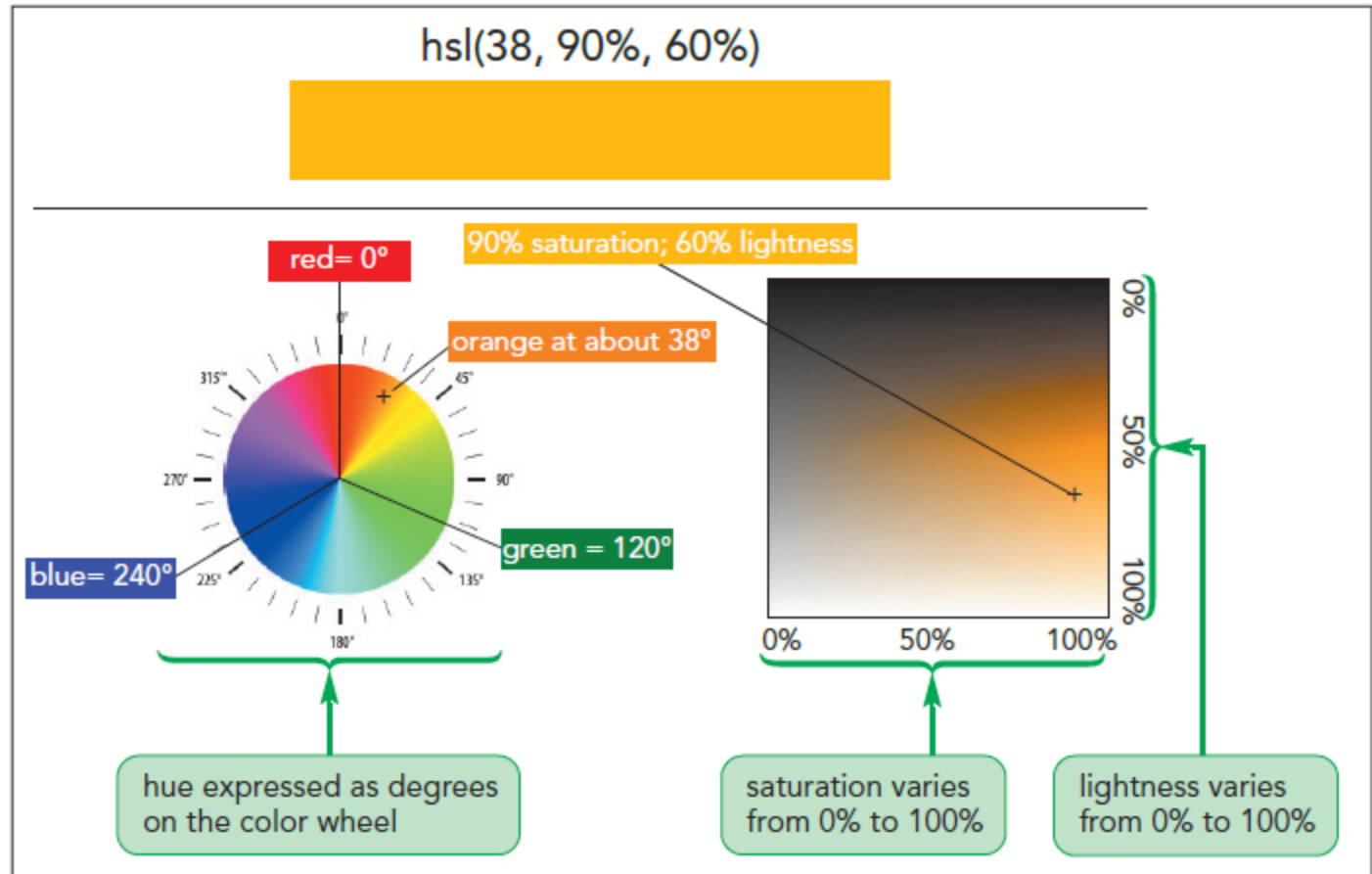
HSL Color Values

- **Hue** – Tint of a color, represented by a direction on a color wheel
- **Saturation** – Measures the intensity of a color and ranges from 0% (no color) up to 100% (full color)
- **Lightness** – Measures the brightness of a color and ranges from 0% (black) up to 100% (white)

HSL Color Values (continued)

Figure 2-9

Defining the color orange under the HSL color model



© 2016 Cengage Learning

Defining Semi-Opaque Colors

- **Opacity** – Defines how solid a color appears
- A color's opacity is specified using the following properties:

- `rgba(red, green, blue, opacity)`
- `hsla(hue, saturation, lightness, opacity)`

where `opacity` sets the opacity of the color ranging from 0 (completely transparent) up to 1.0 (completely opaque)

Setting Text and Background Color

- CSS defines the text and background color for each element on a webpage

`color: color;`

`background-color: color;`

where *color* is a color name or a color value

Employing Progressive Enhancement

- **Progressive enhancement** – A technique of placing the code conforming to older standards before newer properties
- It provides support for older browsers and allows newer standards to be used by the browsers that support them

Contextual Selectors

- **Contextual selector** – Specifies the context under which a particular page element is matched
- Context is based on the hierarchical structure of a document, which involves the relationships between a **parent element** containing one or more **child elements** and within those child elements several levels of **descendant elements**

Contextual Selectors (continued 1)

Contextual selectors

Selector	Description
<code>*</code>	Matches any element
<code>elem</code>	Matches the element <i>elem</i> located anywhere in the document
<code>elem1, elem2, ...</code>	Matches any of the elements <i>elem1</i> , <i>elem2</i> , etc.
<code>parent descendant</code>	Matches the <i>descendant</i> element that is nested within the <i>parent</i> element at some level
<code>parent > child</code>	Matches the <i>child</i> element that is a child of the <i>parent</i> element
<code>elem1 + elem2</code>	Matches <i>elem2</i> that is immediately preceded by the sibling element <i>elem1</i>
<code>elem1 ~ elem2</code>	Matches <i>elem2</i> that follows the sibling element <i>elem1</i>

Contextual Selectors (continued 2)

- To match any element, a **wildcard selector** with the * character is used
- **Sibling selectors** are used to select elements based on elements that are adjacent to them in the document hierarchy

Attribute Selectors

- Selectors also can be defined based on attributes and attribute values within elements
 - **id** – Identifies specific elements within the document
 - **class** – Identifies a group of elements that share a similar characteristic or property

Attribute Selectors (continued)

Figure 2-15 Attribute selectors

Selector	Selects	Example	Selects
<i>elem#id</i>	Element <i>elem</i> with the ID value <i>id</i>	<code>h1#intro</code>	The h1 heading with the id <i>intro</i>
<i>#id</i>	Any element with the ID value <i>id</i>	<code>#intro</code>	Any element with the id <i>intro</i>
<i>elem.class</i>	All <i>elem</i> elements with the <i>class</i> attribute value <i>class</i>	<code>p.main</code>	All paragraphs belonging to the <i>main</i> class
<i>.class</i>	All elements with the class value <i>class</i>	<code>.main</code>	All elements belonging to the <i>main</i> class
<i>elem[att]</i>	All <i>elem</i> elements containing the <i>att</i> attribute	<code>a[href]</code>	All hypertext elements containing the <i>href</i> attribute
<i>elem[att="text"]</i>	All <i>elem</i> elements whose <i>att</i> attribute equals <i>text</i>	<code>a[href="top.html"]</code>	All hypertext elements whose <i>href</i> attribute equals <i>top.html</i>
<i>elem[att~="text"]</i>	All <i>elem</i> elements whose <i>att</i> attribute contains the word <i>text</i>	<code>a[rel~="glossary"]</code>	All hypertext elements whose <i>rel</i> attribute contains the word <i>glossary</i>
<i>elem[att ="text"]</i>	All <i>elem</i> elements whose <i>att</i> attribute value is a hyphen-separated list of words beginning with <i>text</i>	<code>p[id "first"]</code>	All paragraphs whose <i>id</i> attribute starts with the word <i>first</i> in a hyphen-separated list of words
<i>elem[att^="text"]</i>	All <i>elem</i> elements whose <i>att</i> attribute begins with <i>text</i> [CSS3]	<code>a[rel^="prev"]</code>	All hypertext elements whose <i>rel</i> attribute begins with <i>prev</i>
<i>elem[att\$="text"]</i>	All <i>elem</i> elements whose <i>att</i> attribute ends with <i>text</i> [CSS3]	<code>a[href\$="org"]</code>	All hypertext elements whose <i>href</i> attribute ends with <i>org</i>
<i>elem[att*="text"]</i>	All <i>elem</i> elements whose <i>att</i> attribute contains the value <i>text</i> [CSS3]	<code>a[href*="faq"]</code>	All hypertext elements whose <i>href</i> attribute contains the text string <i>faq</i>

Working with Fonts

- **Typography** is the art of designing the appearance of characters and letters on a page
- Color and font are one of few properties in the CSS family of typographical styles

Choosing a Font

- Text characters are based on **fonts** that define the style and appearance of each character in the alphabet
- The general structure of defining font for any page element is

`font-family: fonts;`

where *fonts* is a comma-separated list,
also known as a **font stack**

Choosing a Font (continued)

- **Specific font** – Identified by name and based on a font definition file stored in a user's computer or accessible on the web
- **Generic font** – Describes the general appearance of the characters in the text but does not specify any particular font definition file
 - Supports the font groups **serif, sans-serif, monospace, cursive, and fantasy**

Styling Web Page Text

Figure 2-19 Web safe font stacks

Arial

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: Arial, Helvetica, sans-serif;

Arial Black

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: 'Arial Black', Gadget, sans-serif;

Century Gothic

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: 'Century Gothic', sans-serif;

Comic Sans MS

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: 'Comic Sans MS', cursive;

Courier New

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: 'Courier New', Courier, monospace;

Georgia

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: Georgia, serif;

Impact

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: Impact, Charcoal, sans-serif;

Lucida Console

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: 'Lucida Console', Monaco, monospace;

Lucida Sans Unicode

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: 'Lucida Sans Unicode', 'Lucida Grande', sans-serif;

Palatino Linotype

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: 'Palatino Linotype', 'Book Antiqua', Palatino, serif;

Tahoma

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: Tahoma, Geneva, sans-serif;

Times New Roman

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: 'Times New Roman', Times, serif;

Trebuchet MS

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: 'Trebuchet MS', Helvetica, sans-serif;

Verdana

abcdefghijklmnopqrstuvwxyz/1234567890
font-family: Verdana, Geneva, sans-serif;

Exploring Web Fonts

- **Web font** – Definition font is supplied to the browser in an external file because web safe fonts limit the number of fonts choices

Figure 2-22 Web font formats

Format	Description	Browser
Embedded OpenType (EOT)	A compact form of OpenType fonts designed for use as embedded fonts in style sheets	IE
TrueType (TTF)	Font standard used on the Mac OS and Microsoft Windows operating systems	IE, Firefox, Chrome, Safari, Opera
OpenType (OTF)	Font format built on the TrueType format developed by Microsoft	IE, Firefox, Chrome, Safari, Opera
Scalable Vector Graphics (SVG)	Font format based on an XML vocabulary designed to describe resizable graphics and vector images	Chrome, Safari
Web Open Font Format (WOFF)	The W3C recommendation font format based on OpenType and TrueType with compression and additional metadata	IE, Firefox, Chrome, Safari, Opera

The @font-face Rule

- To access and load a web font, add the `@font-face` rule to the style sheet
- Once a web font is defined using the `@font-face` rule, it is included in the font stack

The @font-face Rule (continued 1)

- The general syntax to include @font-face is

```
@font-face {  
    font-family: name;  
    src: url ('url1') format  
        ('text1'),  
        url ('url2') format ('text2'),  
    ...;  
    descriptor1: value1;  
    descriptor2: value2;  
    ...  
}
```

The @font-face Rule (continued 2)

where `name` is the name of the font, `url` is the location of the font definition file, `text` is an optional text description of the font format, and the `descriptor1: value1;` pairs are optional style properties of the font

Setting the Font Size

- To set a font size, use the style property

```
font-size: size;
```

where *size* is a CSS unit of length in either relative or absolute units.

- **Absolute units** – Fixed in size regardless of the output device and are used only with printed media
- **Relative units** – Expressed relative to the size of other objects within the web page or to the display properties of the device itself

Scaling Fonts with ems and rems

- Text is made scalable with all font sizes expressed relative to default font sizes
- The three relative measurements used to provide scalability are
 - percentage
 - em unit
 - rem or root em unit

Using Viewport Units

- **Viewport unit** – A relative unit used to express length as a percentage of the width and height of the browser window
- CSS3 introduced four viewport units
 - 1 vw = 1% of the browser window width
 - 1 vh = 1% of the browser window height
 - 1 vmin = 1 vw or 1 vh (whichever is smaller)
 - 1 vmax = 1 vw or 1 vh (whichever is larger)

Sizing Keywords

- Font sizes are expressed using the following keywords
 - `xx-small`
 - `x-small`
 - `small`
 - `medium`
 - `large`
 - `x-large`
 - `xx-large`
 - `larger`
 - `smaller`

Controlling Spacing and Indentation

- **Kerning** measures the amount of space between characters, while **tracking** measures the amount of space between words
- The properties to control an element's kerning and tracking are:

```
letter-spacing: value;  
word-spacing: value;
```

Controlling Spacing and Indentation (continued 1)

- **Leading** – Measures the amount of space between lines of text and is set using the following line-height property:
- Text spacing can be controlled by setting the indentation for the first line of text block by using the **text-indent** property

```
line-height: size;
```

```
text-indent: size;
```

Controlling Spacing and Indentation (continued 2)

Figure 2-27

Controlling letter spacing and line height

```
h1, h2 {  
  font-family: Quicksand, Verdana, Geneva, sans-serif;  
  letter-spacing: 0.1em;  
}
```

sets the space between letters to 0.1em

```
/* Navigation Styles */
```

```
nav {  
  font-size: 0.8em;  
}
```

```
nav > ul {  
  line-height: 2em;  
}
```

double spaces the list of hypertext links

Working with Font Styles

- To specify the font style, use
`font-style: type;`
where *type* is normal, italic, or oblique
- To change the weight of the font, use
`font-weight: weight;`
where *weight* is the level of bold formatting applied to the text

Working with Font Styles

(continued 1)

- To specify a text decoration, use
`text-decoration: type;`
where *type* is none, underline, overline, or line-through
- To transform text, use
`text-transform: type;`
where *type* is capitalize, uppercase, lowercase, or none

Working with Font Styles

(continued 2)

- To display a font variant of text, use
`font-variant: type;`
where *type* is normal or small-caps

Aligning Text Horizontally and Vertically

- To horizontally align the text , use
`text-align: alignment;`
where *alignment* is left, right, center, or justify
- To vertically align the text within each line, use
`vertical-align: alignment;`
where *alignment* is baseline, bottom, middle, sub, super, text-bottom, text-top, or top

Aligning Text Horizontally and Vertically (continued)

Figure 2-28 Values of the vertical-align property

Value	Description
baseline	Aligns the baseline of the element with the baseline of the parent element
bottom	Aligns the bottom of the element with the bottom of the lowest element in the line
middle	Aligns the middle of the element with the middle of the surrounding content in the line
sub	Subscripts the element
super	Superscripts the element
text-bottom	Aligns the bottom of the element with the bottom of the text in the line
text-top	Aligns the top of the element with the top of the text in the line
top	Aligns the top of the element with the top of the tallest object in the line

Combining All Text Formatting in a Single Style

- The text and font styles can be combined using the following shorthand font property:

```
font: style variant weight  
size/height family;
```

where *style* is the font's style, *variant* is the font variant, *weight* is the font weight, *size* is the font size, *height* is the height of each line, and *family* is the font stack

Combining All Text Formatting in a Single Style (continued)

Figure 2-29 Style rule for the body footer

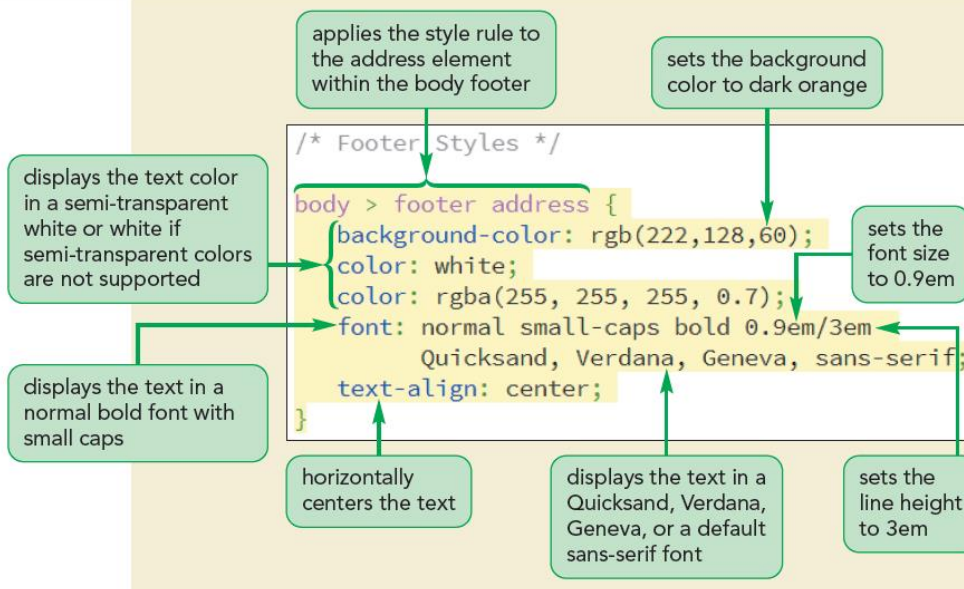
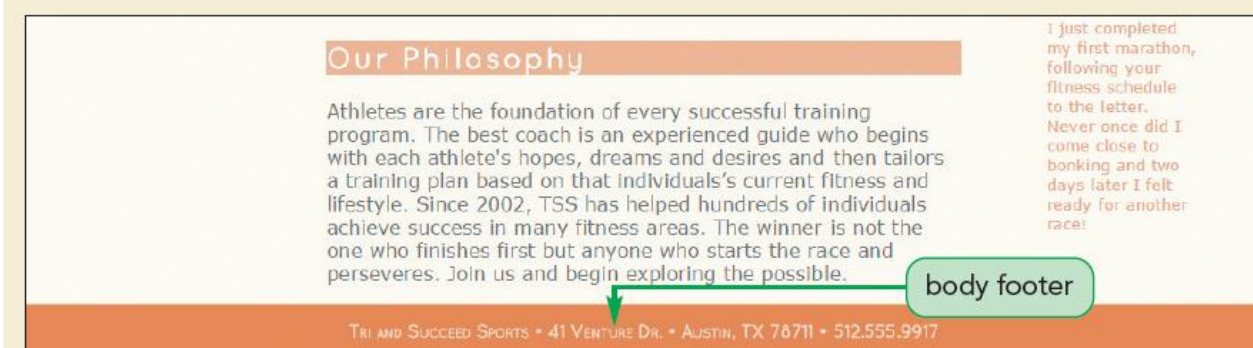


Figure 2-30 Formatted body footer



Formatting Lists

- **List marker** – It is the default browser style symbol displayed before each list item for unordered and ordered lists
- To change the type of list marker or to prevent any display of a list marker, use

`list-style-type: type;`

where *type* is the various types of markers

Formatting Lists (continued)

Figure 2-31 Values of the list-style-type property

list-style-type	Marker(s)
disc	●
circle	○
square	■
decimal	1, 2, 3, 4, ...
decimal-leading-zero	01, 02, 03, 04, ...
lower-roman	i, ii, iii, iv, ...
upper-roman	I, II, III, IV, ...
lower-alpha	a, b, c, d, ...
upper-alpha	A, B, C, D, ...
lower-greek	α, β, γ, δ, ...
upper-greek	Α, Β, Γ, Δ, ...
none	no marker displayed

Using Images for List Markers

- A customized graphic image for the list marker can be supplied by the user

```
list-style-image: url (url);
```

where *url* is the URL of a graphic file containing the marker image

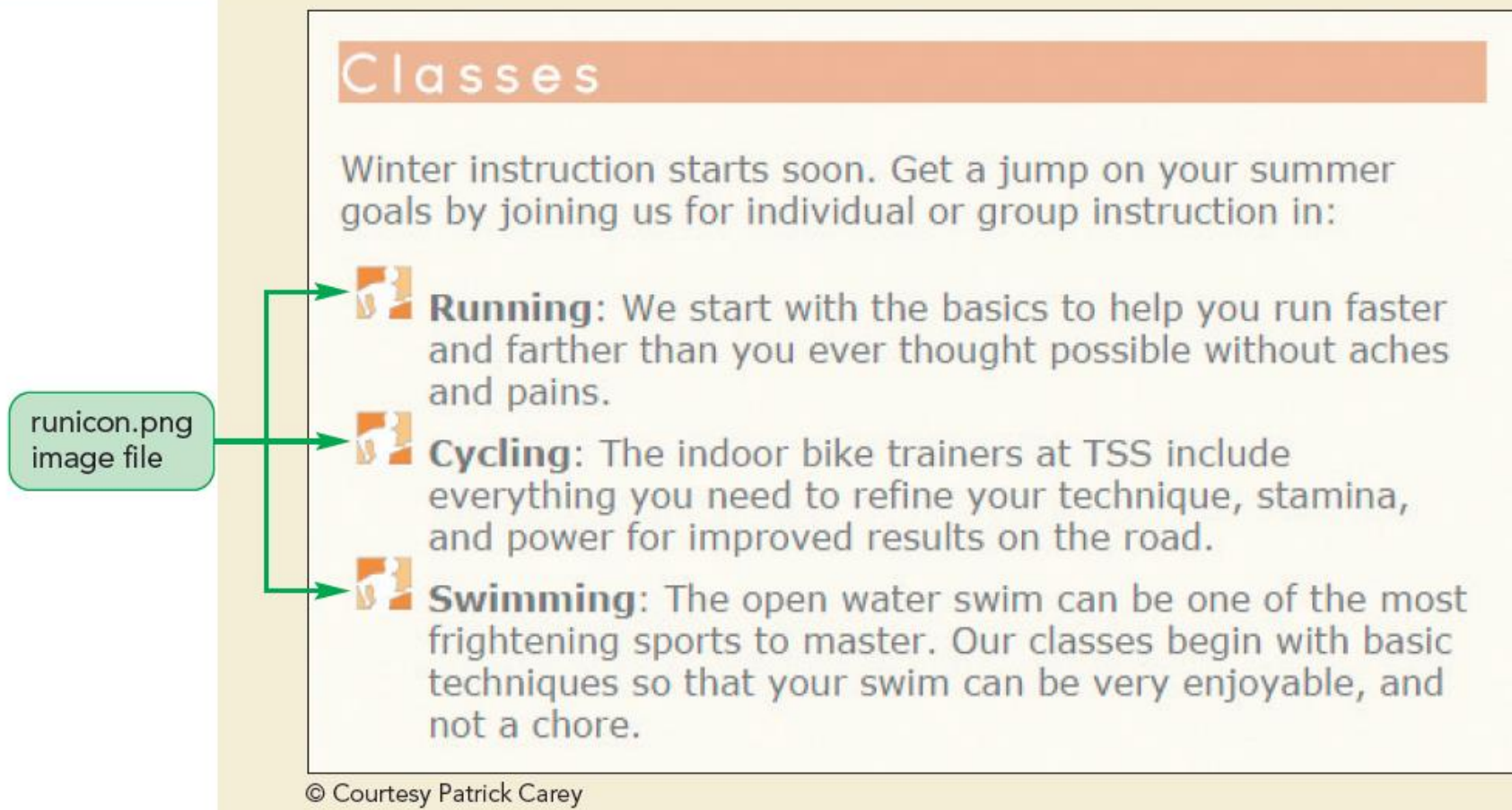
Figure 2-35 Displaying an image in place of a list marker



Using Images for List Markers (continued)

Figure 2-36

Unordered list with the runicon.png image marker



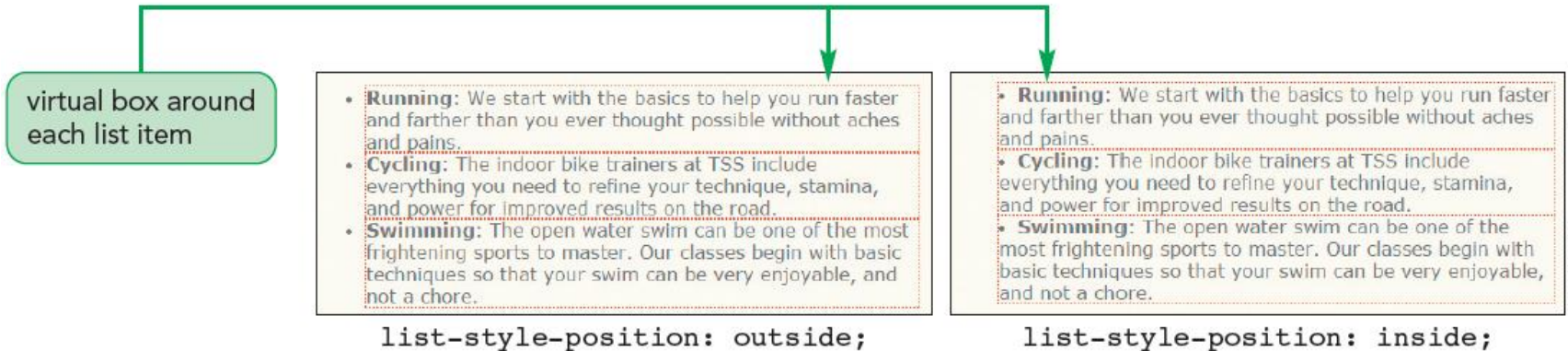
Setting the List Marker Position

- CSS treats each list item as a block-level element, placed within a virtual box in which the list marker is placed outside of the list text
- To change the default behaviour, use `list-style-position: position;` where *position* is either outside or inside

Setting the List Marker Position (continued)

Figure 2-37

Values of the list-style-position property



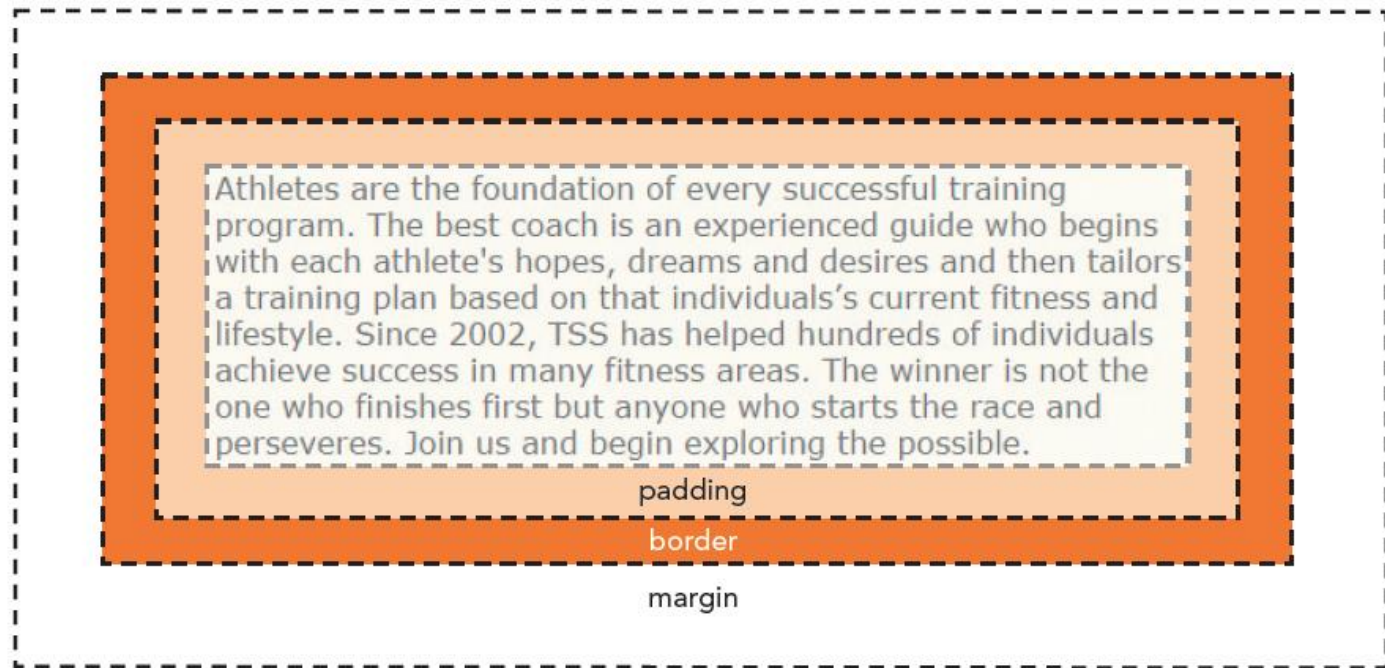
Working with Margins and Padding

- Block-level elements follow the structure of the **box model**
- Contents in a **box model** are enclosed within the following series of concentric boxes:
 - The content of the element itself
 - The **padding space**, which extends from the element's content to a border

Working with Margins and Padding (continued)

- The **border** surrounding the padding space
- The **margin space** comprised of the space beyond the border up to the next page element

Figure 2-38 The CSS box model



© 2016 Cengage Learning

Setting the Padding Space

- To set the width of the padding space, use the following **padding** property

```
padding: size;
```

where *size* is expressed in one of the CSS units of length or the keyword `auto` to let the browser automatically choose the padding

Setting the Padding Space (continued)

Figure 2-39

Setting the size of the left padding space

selects unordered
lists within the nav
element

```
nav > ul {  
  line-height: 2em;  
  list-style-type: none;  
  padding-left: 5px;  
}
```

sets the padding
on the left edge
to 5 pixels

Setting the Margin and Border Spaces

- To set the size of the margin around block-level elements, use
 - `margin: size;`
 - `margin: top right bottom left;`
- To set the size of the border space, use
 - `border-width: size;`
 - `border-width: top right bottom left;`

Setting the Margin and Border Spaces (continued)

Figure 2-40 Setting the size of the top margin

```
nav > ul {  
  line-height: 2em;  
  list-style-type: none;  
  padding-left: 5px;  
}  
  
nav > ul > li.newgroup {  
  margin-top: 20px;  
}
```

selects the list items belonging to the newgroup class found within the unordered navigation list

sets the margin space on the top edge to 20 pixels

Using Pseudo-Classes and Pseudo-Elements

- **Pseudo-class** – classifies an element based on its current status, position, or use in the document

element: pseudo-class

where `element` is an element from the document and `pseudo-class` is the name of a css pseudo-class

- **Structural pseudo-class** – classifies an element based on its location within the structure of the HTML document

Using Pseudo-Classes and Pseudo-Elements (continued 1)

Figure 2-43 Structural pseudo-classes

Pseudo-Class	Matches
<code>:root</code>	The top element in the document hierarchy (the <code>html</code> element)
<code>:empty</code>	An element with no content
<code>:only-child</code>	An element with no siblings
<code>:first-child</code>	The first child of the parent element
<code>:last-child</code>	The last child of the parent element
<code>:first-of-type</code>	The first descendant of the parent that matches the specified type
<code>:last-of-type</code>	The last descendant of the parent that matches the specified type
<code>:nth-of-type(<i>n</i>)</code>	The <i>n</i> th element of the parent of the specified type
<code>:nth-last-of-type(<i>n</i>)</code>	The <i>n</i> th from the last element of the parent of the specified type
<code>:only-of-type</code>	An element that has no siblings of the same type
<code>:lang(<i>code</i>)</code>	The element that has the specified language indicated by <i>code</i>
<code>:not(<i>selector</i>)</code>	An element not matching the specified <i>selector</i>

Using Pseudo-Classes and Pseudo-Elements (continued 2)

Figure 2-44 Applying pseudo-classes to list items



Pseudo-classes for Hypertext

- **Dynamic pseudo-class** – A type of pseudo-class in which the class can change state based on the actions of the user

Figure 2-46 Dynamic pseudo-classes

Pseudo-Class	Description
<code>:link</code>	The link has not yet been visited by the user.
<code>:visited</code>	The link has been visited by the user.
<code>:active</code>	The element is in the process of being activated or clicked by the user.
<code>:hover</code>	The mouse pointer is hovering over the element.
<code>:focus</code>	The element is receiving the focus of the keyboard or mouse pointer.

Pseudo-Elements

- **Pseudo-element** – An object that exists only in the rendered page
- Pseudo-elements can be selected using the following CSS selector:

element::pseudo-element

where *element* is an element from the HTML file and *pseudo-element* is the name of a CSS pseudo-element

Pseudo-Elements (continued)

Figure 2-49 Pseudo-elements

Pseudo-Element	Description
<code>::first-letter</code>	The first letter of the element text
<code>::first-line</code>	The first line of the element text
<code>::before</code>	Content inserted directly before the element
<code>::after</code>	Content inserted directly after the element

Generating Content with CSS

- New content can be added either before or after an element using the following **before** and **after** pseudo-elements:

```
element::before {content: text;
```

```
element::after {content: text;
```

where *text* is the content to be inserted into the rendered web page

Generating Content with CSS (continued)

Figure 2-50 Values of the content property

Value	Description
<code>none</code>	Sets the content to an empty text string
<code>counter</code>	Displays a counter value
<code>attr(<i>attribute</i>)</code>	Displays the value of the selector's <i>attribute</i>
<code>text</code>	Displays the specified <i>text</i>
<code>open-quote</code>	Displays an opening quotation mark
<code>close-quote</code>	Displays a closing quotation mark
<code>no-open-quote</code>	Removes an opening quotation mark, if previously specified
<code>no-close-quote</code>	Removes a closing quotation mark, if previously specified
<code>url(<i>url</i>)</code>	Displays the content of the media (image, video, etc.) from the file located at <i>url</i>

Displaying Attribute Values

- The content property can be used to insert an attribute value into the rendered web page using the **attr()** function

`content: attr(attribute);`

where *attribute* is an attribute of the selected element

Inserting Quotation Marks

- The **blockquote** and **q** elements are used for quoted material
- Decorative opening and closing quotation marks can be inserted using the **content** property as follows:

```
content: open-quote;  
content: close-quote;
```


Inserting Quotation Marks (continued)

Figure 2-51 Adding quotation marks to block quotes

